

WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student

Name: Jack Hanchin

Email: <mailto:hanchijd@mail.uc.edu>



Figure 1: Jack's Headshot

Git: <https://github.com/Hanchijd/Hanchijd.github.io/tree/main/Project2>

Demo Video: <https://youtu.be/EoErwZzKqzs>

Individual Project 2: Secure Full-stack Web Application Development

Project Overview

In this individual project, you will extend your knowledge and skills gained from Labs 3 and 4 to develop a full-stack web application using PHP and MySQL. The project involves creating a simple yet secure login system encompassing user registration, login functionality, profile viewing and editing, and password management. Emphasis will be placed on implementing robust security measures to ensure the integrity and confidentiality of user data. Requirements and grading distribution are outlined below.

Requirements, guidelines, and tutorials were introduced in Lectures 17-18. Make sure that you follow the lectures for detailed instructions. Slides from these lectures are combined and attached for reference. Requirements with grading distribution are outlined below.

******This project demonstrates the application of the course, namely the combination of the student's skills learned from Lab 3 and Lab 4. Said skills were utilized to create a full-stack PHP web application implementing security, user authentication, session management, as well as profile functionality.

Functional Requirements

- **(15 pts) User Registration:** Develop a user registration system that allows new users to create accounts by providing a username, password, name, and email address. Implement both client-side and server-side input validation to ensure data integrity.

A user registration system was created to allow the creation of new accounts from the browser, remotely. This registration system ties each account to a username, password, name and email address. Input validation is added to both the client and server-side to prevent injection as well as increase the security for individual accounts via requiring a minimum length, at least one lowercase letter, one uppercase letter, a number and a special character (i.e. !, @, or #). Server-side securities will be discussed further on in the "Security and Non-technical Requirements section. The code for the front-end is shown below.

registrationform.php:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Sign Up for a new account</title>
  <script type="text/javascript">
    function displayTime() {
      const options = {
        month: 'short', // 'Jun'
        day: '2-digit', // '24'
        hour: '2-digit', // '07'
        minute: '2-digit', // '03'
        second: '2-digit', // '45'
        hour12: true // 'am'
      };
      const formattedTime = new Date().toLocaleString('en-US', options).replace(/,/,' ');
      document.getElementById('digit-clock').innerHTML = "Current time: " +
        formattedTime;
    }
    setInterval(displayTime,500);
  </script>
</head>
<body>
  <h1>New user registration, WAPH</h1>
  <h2>Jack Hanchin</h2>
  <div id="digit-clock"></div>
<?php
```

```

        echo "Visited time: " . date("M-d h:i:sa")
?>
    <form action="addnewuser.php" method="POST" class="form login">
Username:
<input type="text" name="username" required pattern="\w+"
        title="Letters, numbers, and underscores only"><br>

Password:
<input type="password" name="password" required
        pattern="^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#\$%^&]).{8,}$"
        title="At least 8 characters with 1 uppercase, 1 lowercase, 1 number, and
        1 special symbol"><br>

Retype Password:
<input type="password" name="repassword" required title="Passwords must match"><br>

Full Name:
<input type="text" name="fullname" required pattern="[A-Za-z ]+"
        title="Letters and spaces only"><br>

Email:
<input type="email" name="email" required
        title="Enter a valid email address"><br>

    <button type="submit">Sign Up</button>
</form>
</body>
</html>

```

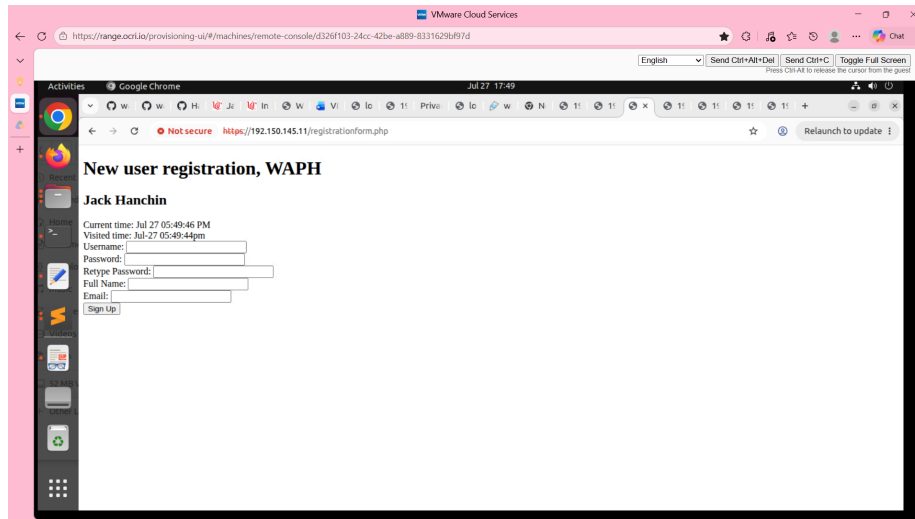


Figure 2: New User Registration Form

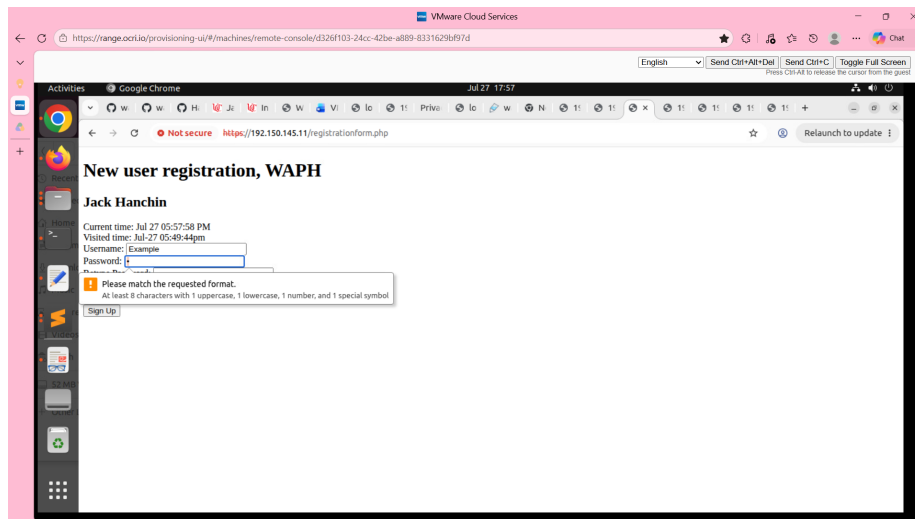


Figure 3: Client-side Password Input Validation

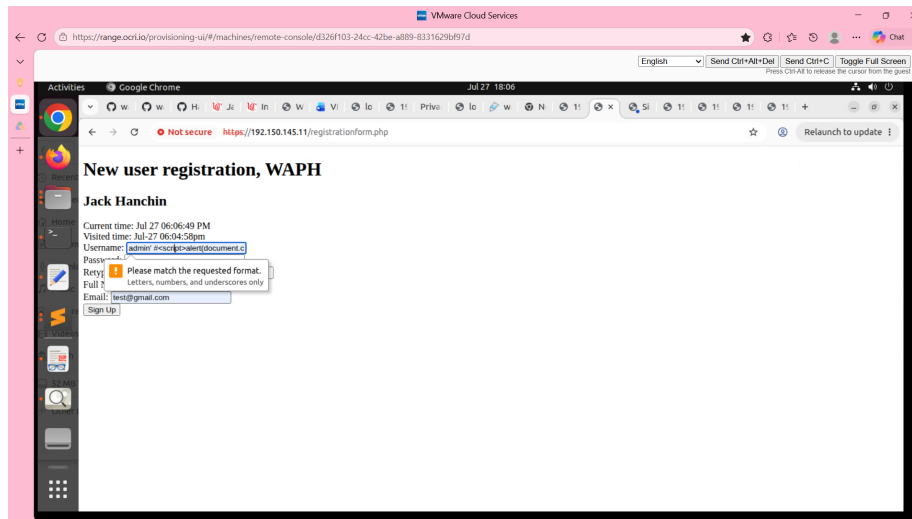


Figure 4: Client-side Username Input Validation

- (15 pts) **Login:** Implement a secure login system that authenticates users and allows them to access their profiles. Use session management to maintain user state across the application.

A secure login system was created that avoids SQL injection, CSRF attacks, Session/Cookie theft, etc. Without having logged in, none of the other pages can be accessed. The code is shown below.

form.php:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>WAPH-Login page</title>
  <script type="text/javascript">
    function displayTime() {
      const options = {
        month: 'short', // 'Jun'
        day: '2-digit', // '24'
        hour: '2-digit', // '07'
        minute: '2-digit', // '03'
        second: '2-digit', // '45'
        hour12: true // 'am'
      };
      const formattedTime = new Date().toLocaleString('en-US', options).replace(/,/ , ' ');
      document.getElementById('digit-clock').innerHTML = "Current time: " +
```

```

        formattedTime;
    }
    setInterval(displayTime,500);
</script>
</head>
<body>
    <h1>A Simple login form, WAPH</h1>
    <h2>Student Name</h2>
    <div id="digit-clock"></div>
<?php
    //some code here
    echo "Visited time: " . date("M-d h:i:sa")
?>
    <form action="index.php" method="POST" class="form login">
        Username:<input type="text" class="text_field" name="username" /> <br>
        Password: <input type="password" class="text_field" name="password" /> <br>
        <button class="button" type="submit">Login</button>
    </form>
</body>
</html>

```

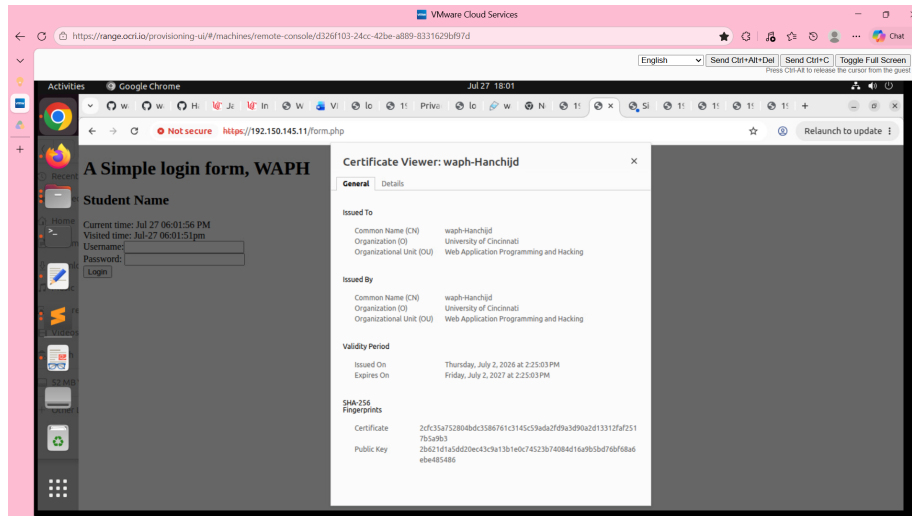


Figure 5: Login Form with HTTPS Certificate

- **(15 pts) Profile Management:** Enable users to view and edit their profile information, such as name and email.

Code for the front-end is shown below.

editprofileform.php:

```
<?php
require "session_auth.php";

if (empty($_SESSION['token'])) {
    $_SESSION['token'] = bin2hex(random_bytes(32));
}

$pass = 'Pa$$w0rd';
$mysqli = new mysqli('localhost', 'Hanchijd', $pass, 'waph');
$sql = "SELECT fullname, email FROM users WHERE username=?";
$stmt = $mysqli->prepare($sql);
$stmt->bind_param("s", $_SESSION['username']);
$stmt->execute();
$stmt->bind_result($fullname, $email);
$stmt->fetch();
$stmt->close();
$mysqli->close();
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Edit Profile</title>
</head>
<body>
    <h2>Edit Profile for <?php echo htmlentities($_SESSION['username']); ?></h2>

    <form action="editprofile.php" method="POST">
        <input type="hidden" name="token" value="<?php echo $_SESSION['token']; ?>">
        <label>Full Name:</label>
        <input type="text" name="fullname" value="<?php echo htmlentities($fullname); ?>" required><br>

        <label>Email:</label>
        <input type="email" name="email" value="<?php echo htmlentities($email); ?>" required><br>

        <button type="submit">Save Changes</button>
    </form>
```



```
<a href="index.php">Back to Profile</a>
</body>
</html>
```

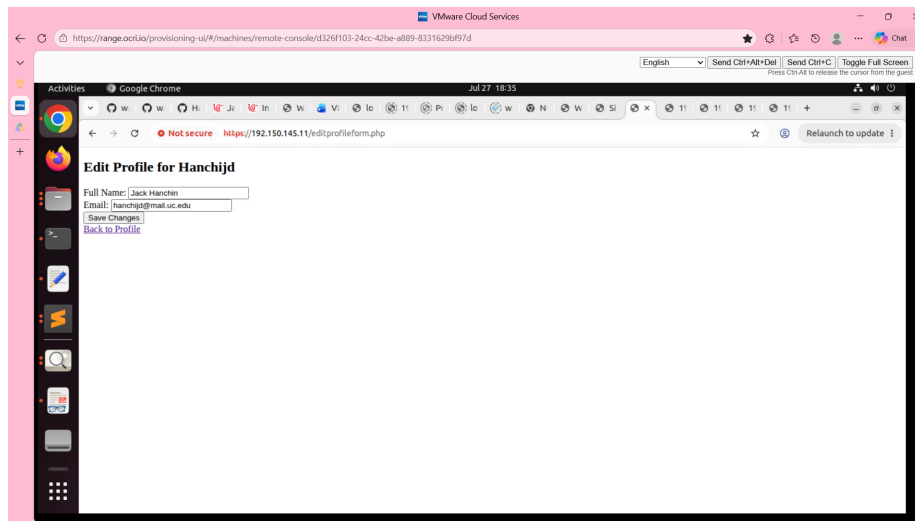


Figure 6: Profile Management Page

- (15 pts) **Password Update:** Allow users to change their passwords securely.

The page for changing passwords allows for users to securely change their password. It can only be accessed after logging in, and to confirm a password change the original password must be input. In addition, to hopefully prevent ‘typos’, the new password must be input twice and must match, as programmed client-side.

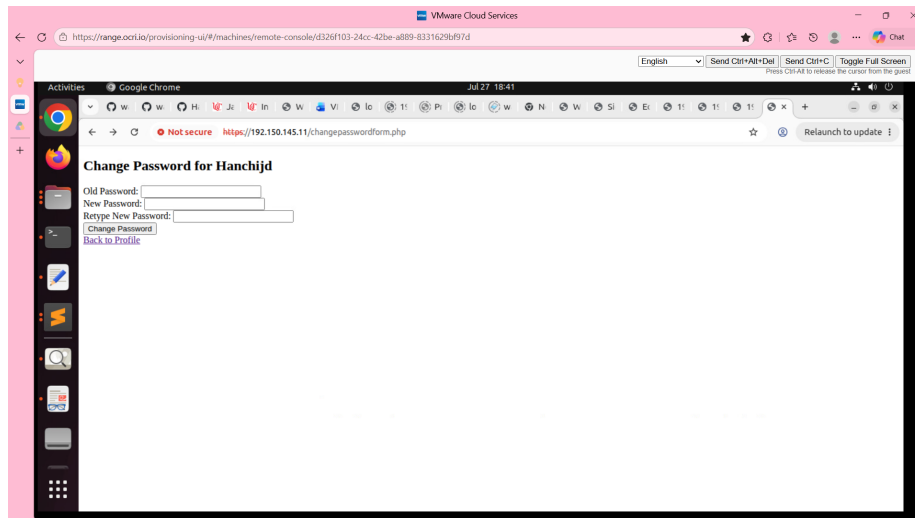


Figure 7: Password Update Page

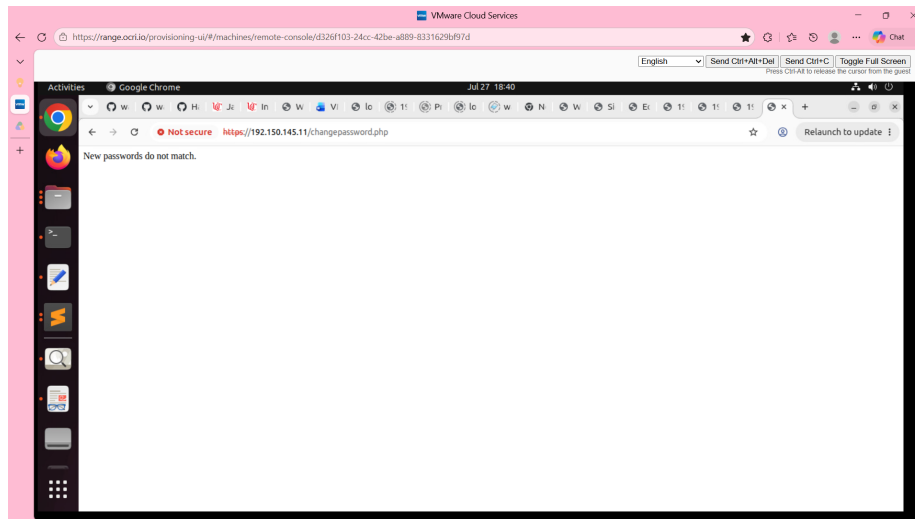


Figure 8: Passwords must match

Security and Non-technical Requirements

- **(5 pts) Security:** The application must be deployed over HTTPS. Passwords must be hashed before being stored in the database. Do not use the MySQL root account in your PHP code. Ensure all SQL operations use prepared statements to mitigate SQL injection attacks.

Example of one of the used prepared statements, using the email input from the `editprofileform.php`: `$sql = "SELECT fullname, email FROM users WHERE username=?";`

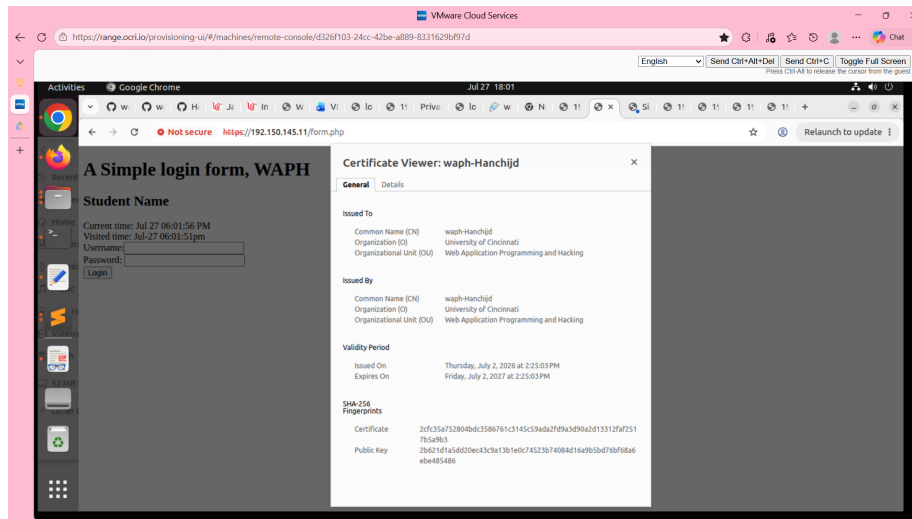


Figure 9: HTTPS Certificate

- (5 pts) **Input Validation:** Implement comprehensive input validation on both the client and server sides to prevent common web vulnerabilities such as XSS attacks.

Password is validated on client-side (HTML5) and must be at least 8 characters long, have an uppercase letter, a lowercase letter, a symbol and a number. The username is likewise validated on the client side and cannot be created with any character beyond letters, numbers and underscores. The Full name is limited to letters and spaces only, and the email must have an @. On the backend, everything utilizes prepared statements to help mitigate web vulnerabilities such as XSS attacks, such as `$sql = "SELECT fullname, email FROM users WHERE username=?";` for the email input and utilization. Also used are empty field checks: `$sql = "SELECT fullname, email FROM users WHERE username=?";`, input sanitization: `$fullname = htmlspecialchars(trim($_POST["fullname"]));`, `$email = htmlspecialchars(trim($_POST["email"]));`, email validation: `if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {die("Invalid email format.");}`, password confirmation, `if ($newpassword !== $renewpassword) {die("New passwords do not match.");}`, old password verification, and CSRF token validation: `if ($token !== $_SESSION['token']) {die("CSRF token mismatch.");}`.

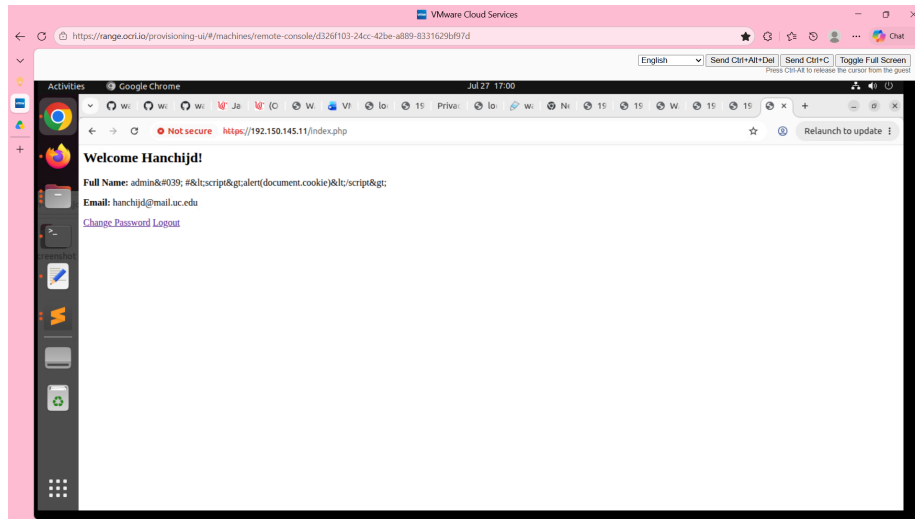


Figure 10: Successful noninvasive pass

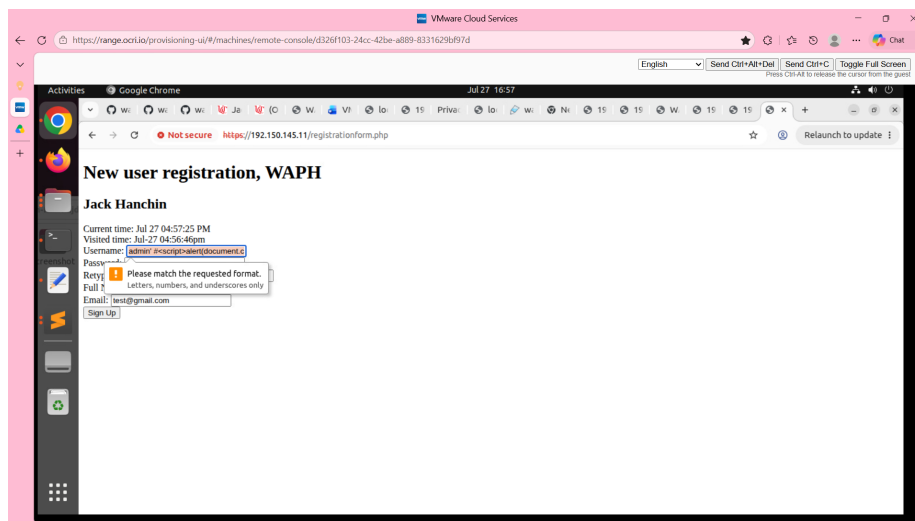


Figure 11: Refused Injection

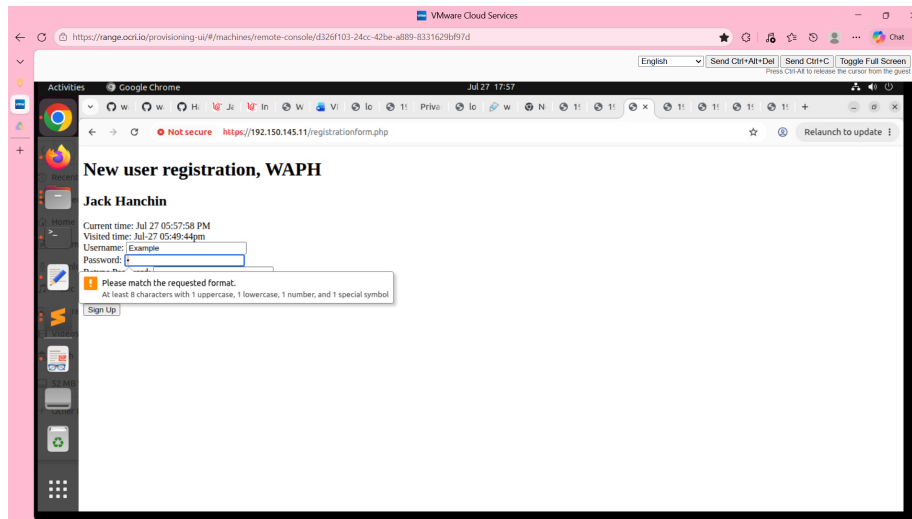


Figure 12: Password Validation/Verification

- (5 pts) **Database Design:** Design and implement a MySQL database to store user information securely. Ensure that database interactions are performed using secure practices.

As shown in the screenshot below, all users are limited to access within the waph database, meaning none have global access beyond this database on the specified ip, in this case localhost. The database was created utilizing the database-data.sql file, as shown:

```
drop table if exists users; create table users(
  username
  varchar(50) PRIMARY KEY,
  password varchar(100) NOT NULL,
  fullname varchar(100) NOT NULL,
  email VARCHAR(100) NOT NULL);
INSERT INTO users(username,password) VALUES ('admin',md5('1234'));
DROP TABLE IF EXISTS users;
```

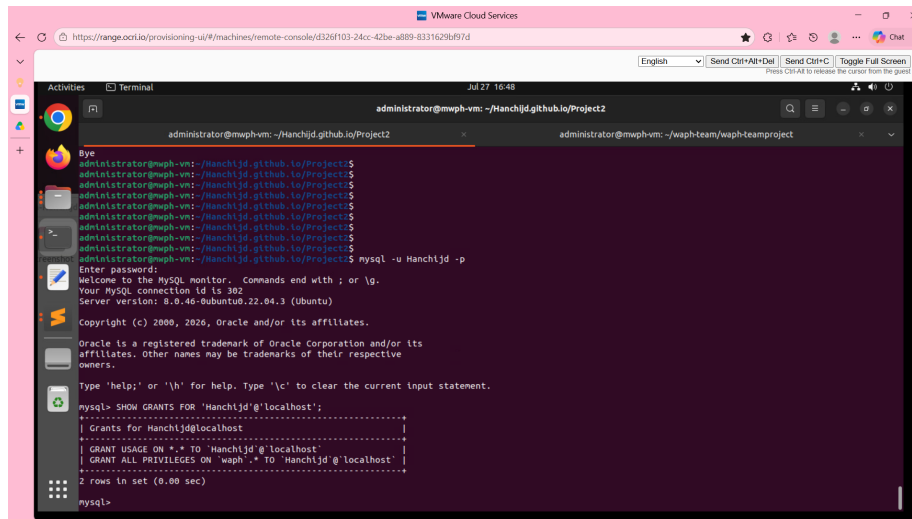


Figure 13: Database permissions

- **(5 pts) Front-end Development:** Use HTML, CSS (with an option to integrate a CSS framework or template), and JavaScript to create an intuitive and responsive user interface. Include necessary client-side validations using HTML5 and JavaScript.

The new password page:

```

<form action="changepassword.php" method="POST">
  <label>Old Password:</label>
  <input type="password" name="oldpassword" required>

  <label>New Password:</label>
  <input type="password" name="newpassword" required>

  <button type="submit">Change Password</button>
</form>

```

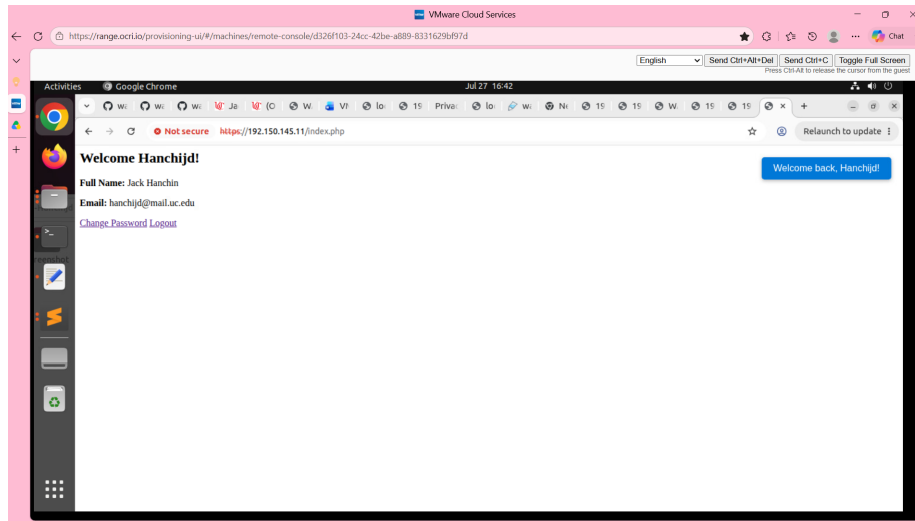


Figure 14: Minimalistic Successful Login Page

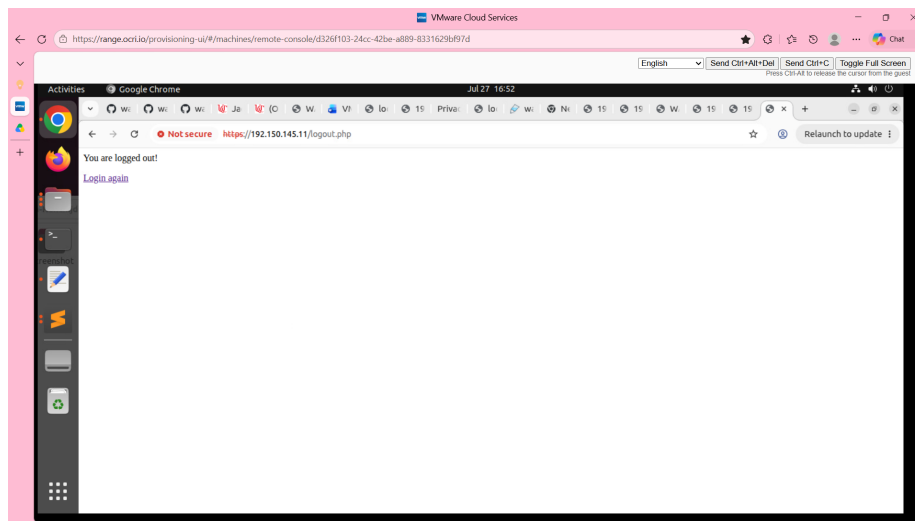


Figure 15: Minimalistic Logout Page

- (5 pts) **Session Management:** Implement secure session management for user authentication. Protect against session hijacking and fixation attacks.

One of the main factors for ensuring session management is one line added to every secure page, require "session_auth.php";, which requires session_auth.php to run and return true. The code is as follows below. The lifetime ensures a login must reoccur after 15 minutes to prevent hijacking. The \$domain forces a matched ip for access. The \$secure ensures security by only sending cookies over HTTPS, never http, while \$httponly forbids JavaScript from reading the user's session cookie, essentially preventing the running (in almost any form) of document.cookie, adding another layer of redundancy for protection. session_auth.php also runs a session authentication to verify login when run, so every time you switch between a page it will destroy the session without a successful login in the past fifteen minutes. Finally, session_auth.php checks the browser session against the HTTP_USER server agent, and if it does not match it also destroys the session.

```
<?php
$lifetime = 15 * 60;
$path = "/";
$domain = "192.150.145.11";
$secure = TRUE;
$httponly = TRUE;

session_set_cookie_params($lifetime, $path, $domain, $secure, $httponly);
session_start();

if (!isset($_SESSION["authenticated"]) || $_SESSION["authenticated"] != TRUE) {
    session_destroy();
    echo "<script>alert('You have not logged in. Please login first');</script>";
    header("Refresh:0; url=form.php");
    die();
}

if ($_SESSION["browser"] != $_SERVER["HTTP_USER_AGENT"]) {
    session_destroy();
    echo "<script>alert('Session hijacking attack detected!');</script>";
    header("Refresh:0; url=form.php");
    die();
}
?>
```

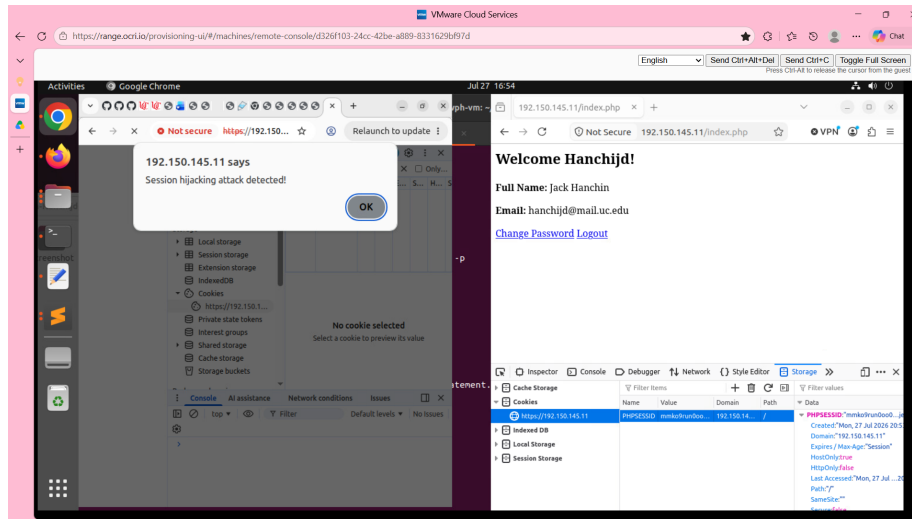


Figure 16: Session Hijacking Protection

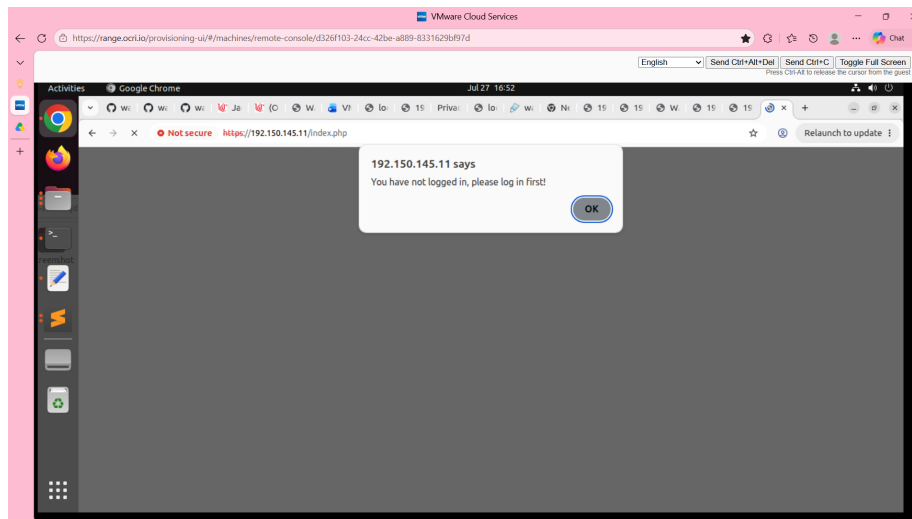


Figure 17: Not Logged In

- **(5 pts) CSRF Protection:** Incorporate mechanisms such as using anti-CSRF tokens to protect against Cross-Site Request Forgery (CSRF) attacks in database modification use cases.

When loading a secure page, such as the page that changes passwords or shows the profile or modifies the profile information, a random CSRF token is generated using: `if (empty($_SESSION['token'])) {$_SESSION['token'] = bin2hex(random_bytes(32)); }`. This is also embedded as a hidden field as: `<input type="hidden" name="token" value="<?php echo $_SESSION['token']; ?>">`. The server then checks via, `if ($token !== $_SESSION['token']) { die("CSRF token mismatch. Please try again."); }`. If the token is missing or mismatched, the request to access the page is rejected, no database changes occur, and an error message is displayed. This was tested in the `CSRF_attack.html` file (shown below). When the CSRF attack was attempted, “CSRF token mismatch. Please try again.” was displayed, showing successful CSRF protection. Screenshots of this attack are below.

```
<form action="http://YOUR_VM_IP/changepassword.php" method="POST">
  <input type="hidden" name="oldpassword" value="anything">
  <input type="hidden" name="newpassword" value="hacked123">
  <input type="hidden" name="renewpassword" value="hacked123">
  <button type="submit">Attack</button>
</form>
```

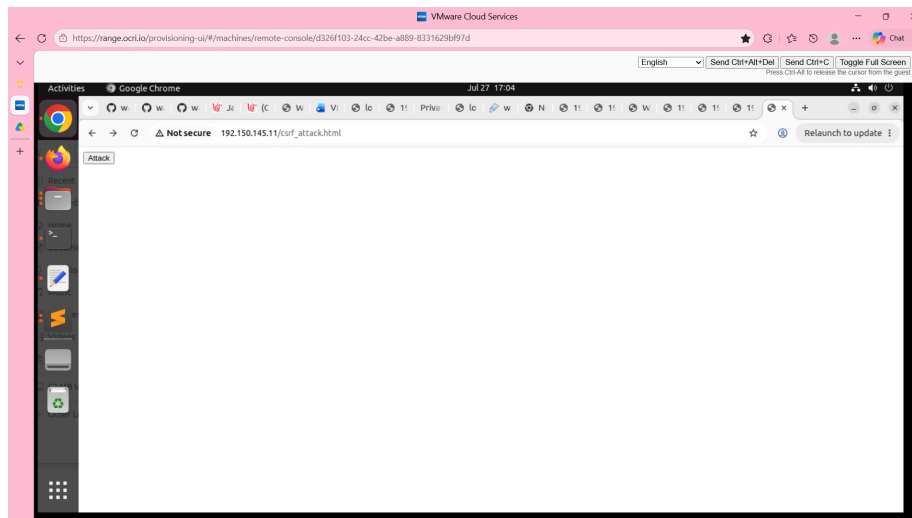


Figure 18: Attack Page

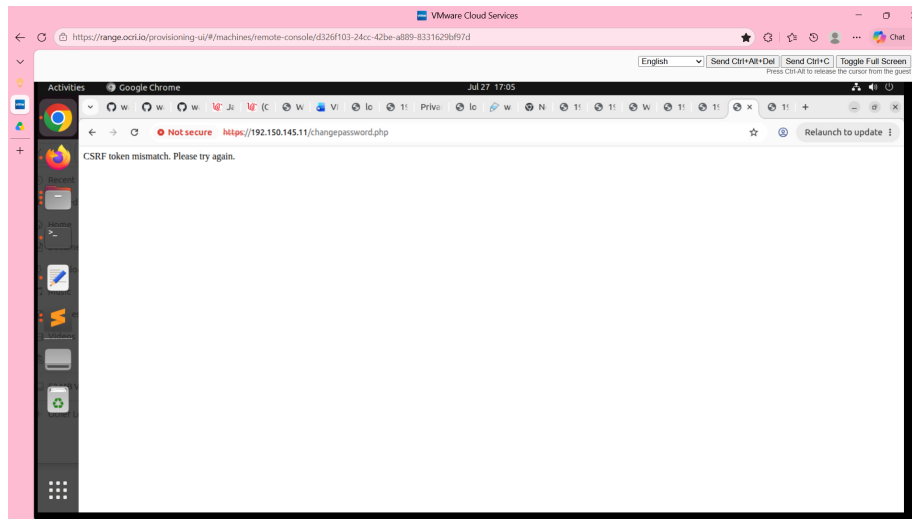


Figure 19: Failed Attack

(10 pts) Deliverables and Report

You must write a report using Markdown format. Your report should follow the template/outline provided in Lecture 2, which should include the course name and instructor, your name and email together with your headshot (150x150 pixels), and sub-sections of the project's overview, and each requirement.

There should be an overview sub-section where you must write an overview of the assignment and the outcomes you learned from it. Include the direct clickable link to the project folder on GitHub.com so that it can be viewed when grading, for example, <https://github.com/Hanchijd/Hanchijd.github.io/tree/main/Project2>. All project code, including HTML, CSS, PHP, and SQL scripts must be available on your private repository and **included in the report as an appendix**.

For each requirement, write a brief summary of how you complete it. You are welcome to include code snippets and screenshots to demonstrate the outcome, however, they are not required.

Demonstration Video: Record a 5-minute video demonstrating the functional requirements implemented for the project and include the video link (uploaded online) in the report for grading.

DEMO VIDEO: <https://youtu.be/EoErwZzKqzs>

Submission

You need to submit two files for grading:

Your report in the PDF mentioned above. The PDF file should be named `your-username-waph-project2.pdf`, e.g., `hanchijd-waph-project2.pdf`.

The source code of the entire project in a compressed ZIP file. The PDF file should be named `your-username-waph-project2.zip`, e.g., `hanchijd-waph-project2.zip`.